

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

**ОТЧЕТ по лабораторной работе №2
по дисциплине «Численные методы»**

**Тема: Интерполяционный многочлен Ньютона
Вариант № 11**

Выполнил:
Студент 3 курса, 7 группы
Стаселько Илья Юрьевич

Преподаватель:
Горбачева Юлия Николаевна

Минск 2026

1 Постановка задачи

Написать программу, которая строит на отрезке $[a, b]$ для данной функции $f(x)$ интерполяционный многочлен $P_n(x)$ в форме Ньютона произвольной степени n по равноотстоящим и чебышевским узлам. Программа должна позволять вычислять значения этих многочленов в заданной точке с применением схемы Горнера.

С помощью написанной программы необходимо провести вычислительный эксперимент:

1. Построить совмещенные графики функций и интерполяционных многочленов $P_n(x)$ для $n = 2, 10, 20$ для каждого типа узлов. Графики строятся по массиву точек $\bar{x}_i$, $i = 0, \dots, 500$.
2. Построить совмещенный график сходимости интерполяционных процессов (в логарифмическом масштабе по оси Oy) в зависимости от степени n (от 1 до 61 с шагом 5). По оси Oy откладывается равномерная норма погрешности $\|r_n\|$.
3. Сделать выводы о сходимости в зависимости от выбора узлов и вида функции.

Функции варианта 11:

$$f_1(x) = \cos(2x), \quad f_2(x) = \sin |x| + 1$$

Отрезок интерполирования: $[a, b] = [-2, 2]$.

2 Краткие теоретические сведения

2.1 Способ выбора узлов интерполирования

Для интерполирования используются два типа сеток узлов:

1. **Равноотстоящие узлы:**

$$x_i = a + i \frac{b-a}{n}, \quad i = 0, \dots, n$$

2. **Чебышевские узлы:**

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos \left(\frac{(2i+1)\pi}{2(n+1)} \right), \quad i = 0, \dots, n$$

2.2 Разделенные разности

Разделенная разность первого порядка определяется как:

$$f(x_i, x_j) = \frac{f(x_j) - f(x_i)}{x_j - x_i}$$

Разделенная разность k -го порядка выражается рекурсивно:

$$f(x_0, \dots, x_k) = \frac{f(x_1, \dots, x_k) - f(x_0, \dots, x_{k-1})}{x_k - x_0}$$

2.3 Интерполяционный многочлен Ньютона и схема Горнера

Интерполяционный многочлен Ньютона записывается с использованием разделенных разностей:

$$P_n(x) = f(x_0) + (x - x_0)f(x_0, x_1) + \dots + \prod_{i=0}^{n-1} (x - x_i)f(x_0, \dots, x_n)$$

Для оптимизации вычислений значения полинома в заданной точке применяется алгоритм схемы Горнера. Метод заключается в последовательном вынесении за скобки биномов $(x - x_i)$, начиная со старшей разделенной разности:

$$P_n(x) = f(x_0) + (x - x_0)\left(f(x_0, x_1) + \dots + (x - x_{n-1})f(x_0, \dots, x_n)\right)$$

3 Вычислительный эксперимент

В ходе эксперимента для функций $f_1(x)$ и $f_2(x)$ были построены интерполяционные многочлены $P_n(x)$.

3.1 Графики функций и интерполяционных многочленов

На рисунке 1 представлены графики исходных функций и построенных интерполяционных многочленов для степеней $n = 2, 10, 20$ на равномерной и чебышевской сетках.

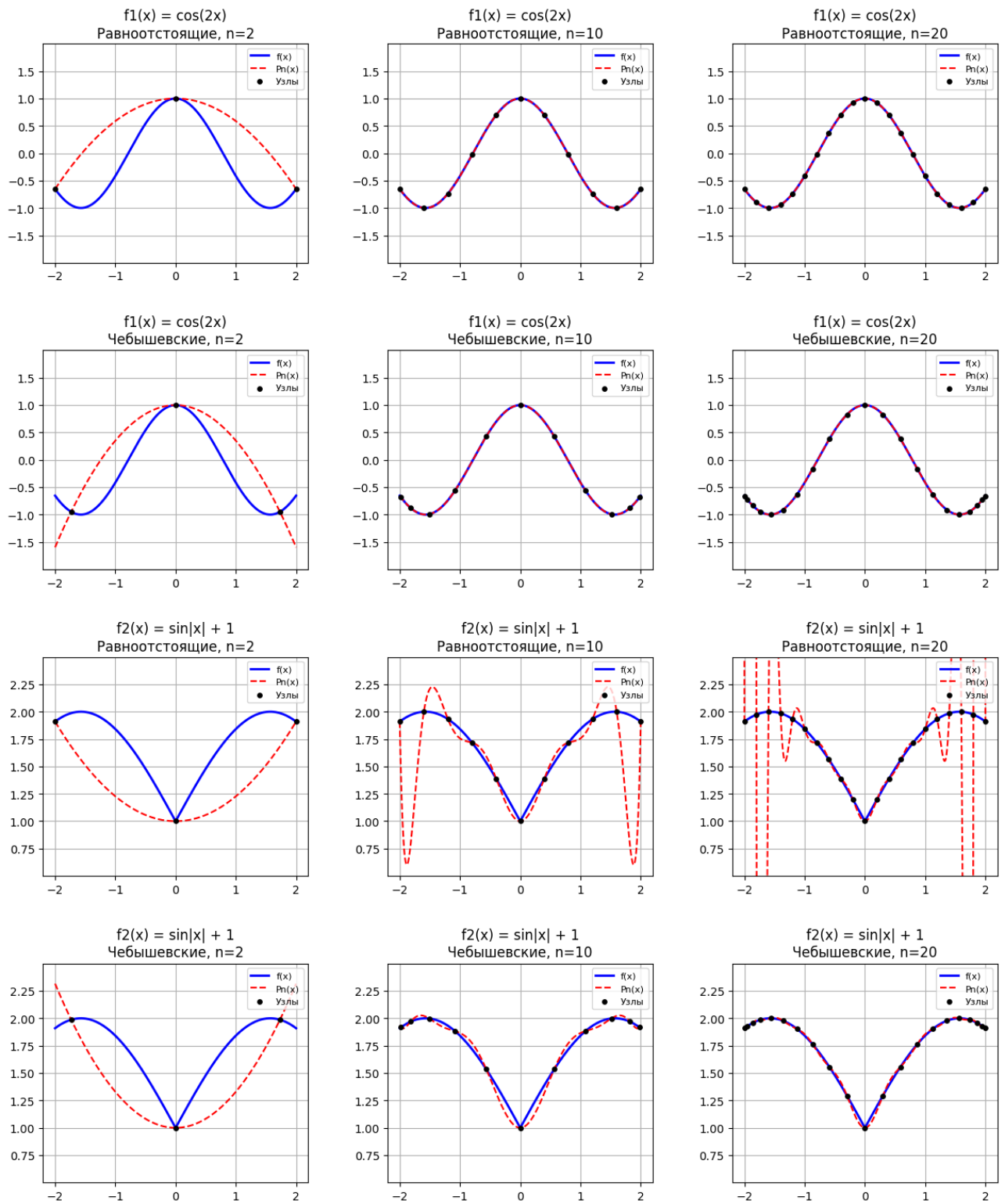


Рис. 1: Совмещенные графики функций $f(x)$ и многочленов $P_n(x)$

3.2 Графики сходимости интерполяционных процессов

На рисунке 2 изображены графики изменения равномерной нормы погрешности $\|r_n\| = \max_i |P_n(\bar{x}_i) - f(\bar{x}_i)|$ в зависимости от степени многочлена n .

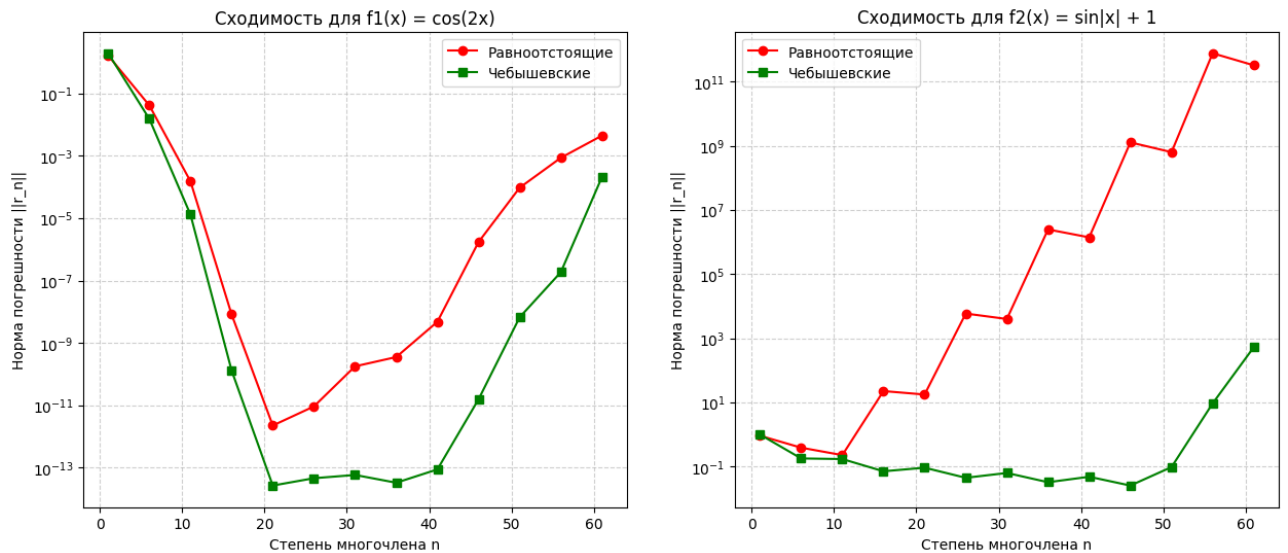


Рис. 2: Графики сходимости интерполяционных процессов (логарифмический масштаб)

4 Выводы

На основании полученных результатов можно сделать следующие выводы о сходимости интерполяционного процесса:

1. **Для гладкой функции $f_1(x) = \cos(2x)$:** Данная функция является бесконечно дифференцируемой (целой). Интерполяционный процесс сходится (погрешность падает) для обоих типов узлов при начальных и средних значениях n . При этом на чебышевской сетке погрешность достигает машинного нуля гораздо надежнее. На равноотстоящих узлах при $n > 30$ ошибка начинает возрастать из-за вычислительной неустойчивости разделенных разностей и накопления ошибок округления.
2. **Для негладкой функции $f_2(x) = \sin|x| + 1$:** Функция имеет излом в точке $x = 0$, что серьезно затрудняет интерполяцию алгебраическими многочленами. На равноотстоящих узлах погрешность стремительно и неограниченно возрастает на концах отрезка.
3. **Сравнение узлов:** Использование **чебышевских узлов** ступцает сетку интерполяции к концам отрезка, что является оптимальным согласно теореме о чебышевском альтернансе. Для негладкой функции $f_2(x)$ чебышевская сетка не дает погрешности экспоненциально расти, демонстрируя свое абсолютное превосходство над равномерной сеткой при интерполяции функций с "плохими" свойствами гладкости.

5 Листинг программы

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import warnings
4
5 # Отключаем предупреждения об overflow, так как при высоких степенях n
6 # вычисление разделенных разностей на равноотстоящей сетке математически неустойчиво
7 warnings.filterwarnings('ignore')
8
9 # Задаем функции варианта 11
10 def f1(x):
11     return np.cos(2 * x)
12
13 def f2(x):
14     return np.sin(np.abs(x)) + 1
15
16 # Отрезок интерполирования
17 A, B = -2.0, 2.0
18
19 # Массив из 501 точки для построения графиков и оценки нормы погрешности
20 x_plot = np.linspace(A, B, 501)
21
22 # ----- Генерация узлов -----
23
24 def get_equidistant_nodes(a, b, n):
25     """Возвращает n+1 равноотстоящих узлов на отрезке [a, b]"""
26     i = np.arange(n + 1)
27     return a + i * (b - a) / n
28
29 def get_chebyshev_nodes(a, b, n):
30     """Возвращает n+1 чебышевских узлов на отрезке [a, b]"""
31     i = np.arange(n + 1)
32     return (a + b) / 2 + (b - a) / 2 * np.cos((2 * i + 1) * np.pi / (2 * (n + 1)))
33
34 # ----- Алгоритм Ньютона -----
35
36 def get_divided_differences(x_nodes, y_nodes):
37     """Вычисляет разделенные разности (коэффициенты многочлена Ньютона)"""
38     n = len(x_nodes)
39     coef = np.zeros([n, n])
40     coef[:, 0] = y_nodes
41
42     for j in range(1, n):
43         for i in range(n - j):
44             coef[i][j] = (coef[i + 1][j - 1] - coef[i][j - 1]) / (x_nodes[i + j] - x_nodes[i])
45
46     return coef[0, :]
47
48 def newton_poly_horner(coef, x_nodes, x):
49     """Вычисляет значение многочлена Ньютона в точках x по схеме Горнера"""
50     n = len(coef) - 1
51     result = coef[n]
52
53     for i in range(n - 1, -1, -1):
54         result = result * (x - x_nodes[i]) + coef[i]
```

```

56     return result
57
58 # ----- Вычислительный эксперимент 1 -----
59 # Построение совмещенных графиков  $P_n(x)$  и  $f(x)$ 
60
61 degrees = [2, 10, 20]
62 functions = [(f1, 'f1(x) = cos(2x)'), (f2, 'f2(x) = sin|x| + 1')]
63 node_types = [('Равноотстоящие', get_equidistant_nodes),
64               ('Чебышевские', get_chebyshev_nodes)]
65
66 fig, axes = plt.subplots(4, 3, figsize=(15, 18))
67 fig.subplots_adjust(hspace=0.4, wspace=0.3)
68
69 plot_row = 0
70
71 for func, func_name in functions:
72     y_plot_true = func(x_plot)
73
74     for node_name, node_func in node_types:
75         for j, n in enumerate(degrees):
76             ax = axes[plot_row, j]
77
78             x_nodes = node_func(A, B, n)
79             y_nodes = func(x_nodes)
80
81             coefs = get_divided_differences(x_nodes, y_nodes)
82             y_poly = newton_poly_horner(coefs, x_nodes, x_plot)
83
84             ax.plot(x_plot, y_plot_true, label='f(x)', color='blue', lw=2)
85             ax.plot(x_plot, y_poly, label=f'Pn(x)', color='red', linestyle='--')
86             ax.scatter(x_nodes, y_nodes, color='black', zorder=5, label='Узлы', s=15)
87
88             # Ограничиваем ось Y
89             y_min, y_max = np.min(y_plot_true), np.max(y_plot_true)
90             margin = (y_max - y_min) * 0.5
91             ax.set_ylim(y_min - margin, y_max + margin)
92
93             ax.set_title(f'{func_name}\n{n}{node_name}, n={n}')
94             ax.grid(True)
95             ax.legend(loc='upper right', fontsize=8)
96
97         plot_row += 1
98
99 plt.show()
100
101 # ----- Вычислительный эксперимент 2 -----
102 # Построение графиков сходимости
103
104 n_values = np.arange(1, 62, 5)
105 fig, axes = plt.subplots(1, 2, figsize=(15, 6))
106
107 for idx, (func, func_name) in enumerate(functions):
108     y_plot_true = func(x_plot)
109
110     errors_eq = []
111     errors_cheb = []
112
113     for n in n_values:

```

```

114     # Равноотстоящие узлы
115     x_nodes_eq = get_equidistant_nodes(A, B, n)
116     coefs_eq = get_divided_differences(x_nodes_eq, func(x_nodes_eq))
117     y_poly_eq = newton_poly_horner(coefs_eq, x_nodes_eq, x_plot)
118     errors_eq.append(np.max(np.abs(y_poly_eq - y_plot_true)))
119
120     # Чебышевские узлы
121     x_nodes_cheb = get_chebyshev_nodes(A, B, n)
122     coefs_cheb = get_divided_differences(x_nodes_cheb, func(x_nodes_cheb
123 ))
124     y_poly_cheb = newton_poly_horner(coefs_cheb, x_nodes_cheb, x_plot)
125     errors_cheb.append(np.max(np.abs(y_poly_cheb - y_plot_true)))
126
127 axes[idx].semilogy(n_values, errors_eq, marker='o', label='Равноотстоящи
128 e', color='red')
129 axes[idx].semilogy(n_values, errors_cheb, marker='s', label='Чебышевские
130 ', color='green')
131
132 axes[idx].set_title(f'Сходимость для {func_name}')
133 axes[idx].set_xlabel('Степень многочлена n')
134 axes[idx].set_ylabel('Норма погрешности ||r_n||')
135 axes[idx].grid(True, which="both", ls="--", alpha=0.6)
136 axes[idx].legend()
137
138 plt.show()

```